VolumeVision: A Real-Time Hand Landmark Pipeline for Gesture Control

Kevin Mota da Costa

October 29, 2026

Abstract

VolumeVision is a real-time computer vision pipeline that detects a hand with a pretrained MediaPipe landmark model, extracts a geometric feature (the thumb–index distance) and converts it into a control signal. The signal drives the volume of a test tone. This document describes the detection model and its outputs, the coordinate handling, the gesture feature, and the real-time system around the model.

1 Approach

The pipeline has one learned component and several engineered ones:

camera frame → hand landmark model → pinch distance → control value → actuator

The landmark model is pretrained and used as provided. The contribution of this project is the integration: frame preprocessing, the gesture feature, the real-time loop, and a configuration layer that keeps hardware out of the code.

2 Hand Landmark Model

Detection uses the MediaPipe HandLandmarker (Tasks API) [2] with the `hand_landmarker.task` model (float16). It follows the two-stage design of MediaPipe Hands [1]:

1. a palm detector (single-shot detector) locates the hand in the full frame;
2. a landmark regressor predicts 21 keypoints inside the cropped hand region.

For each hand the model returns 21 landmarks $\ell_i = (x_i, y_i, z_i)$. The coordinates x_i and y_i are normalized to $[0, 1]$ by image width and height; z_i is a relative depth with the wrist as origin, at a scale similar to x. The task also returns handedness and metric *world landmarks*; only the image landmarks are used here.

The model runs in `VIDEO` mode with one hand and the default confidence thresholds (0.5 for detection, presence and tracking). In this mode the palm detector does not run on every frame: after the first detection, the hand region is derived from the previous frame's landmarks, and the detector is invoked again only when tracking is lost. This is what makes per-frame inference cheap enough for real time on a CPU.

3 Preprocessing and Coordinates

Each frame is resized to a fixed width $W = 520$ (aspect ratio preserved) and mirrored horizontally. Landmarks are converted to pixel coordinates

$$p_i = (\lfloor x_i W \rfloor,\ \lfloor y_i H \rfloor)\,. \tag{1}$$

Frames are passed to the model as RGB images with monotonic millisecond timestamps, as the video running mode requires.

4 Gesture Feature

Landmarks 4 (thumb tip) and 8 (index tip) define the pinch distance

$$d = \|p_4 - p_8\|_2, \tag{2}$$

which is mapped to a control value $v \in [0, 100]$ by a clamped linear function:

$$v = \mathrm{round}\left(100 \cdot \mathrm{clip}\left(\frac{d - d_{\min}}{d_{\max} - d_{\min}},\ 0,\ 1\right)\right), \qquad d_{\min} = 20,\ d_{\max} = 200. \tag{3}$$

If no hand is detected, $v = 0$.

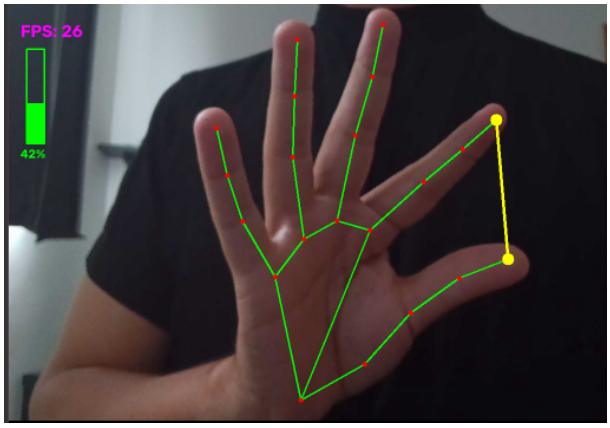


Figure 1: Live view: hand landmarks, the pinch segment p_4p_8 (yellow), the volume bar and the frame rate.

The feature is computed in image pixels, so d depends on the hand-to-camera distance: the same pinch yields a larger d when the hand is closer. Two ways to remove this dependence are available but not implemented: normalizing d by a hand-size reference such as $\|p_0 - p_9\|_2$, or using the metric world landmarks returned by the model.

5 Real-Time System

Each frame is processed in a synchronous loop: read, resize and mirror, detect, compute the feature, update the actuator, draw the overlay, display.

Stage	Module	Role
Input	`hardware/cam.py`	`cv2.VideoCapture`, buffer size 1
Detection	`vision/detector.py`	HandLandmarker, pixel landmarks
Gesture	`vision/gesture.py`	$p_4, p_8 \to d \to v$
Actuator	`hardware/audio.py`	test tone, gain set from v
Output	`display/window.py`	MJPEG over HTTP, or local window
Overlay	`display/overlay.py`	landmarks, pinch line, bar, FPS

The camera source accepts a device index, a device path, an HTTP stream URL or a video file. Inference runs on CPU (XNNPACK delegate); the overlay in Figure 1 shows the frame rate, and end-to-end latency was not measured. In web mode each frame is JPEG-encoded and served as `multipart/x-mixed-replace`, so the display needs no X11 forwarding.

The actuator is a sine tone whose gain is $g = (v/100)\, g_{\max}$ with $g_{\max} = 0.5$. It exists to make the detection observable and testable. The gain is interpolated linearly inside each audio block to avoid clicks; this is secondary to the vision pipeline and could be replaced by any other consumer of v.

6 Configuration and Reproducibility

Behavior is defined in `config.yaml`: model path, frame width, window mode and port, distance range and actuator parameters. Hardware is defined in a profile, a Docker Compose override that sets the camera source, the audio device and the device and socket mounts. The Python code contains no hardware-specific value.

Dependencies are pinned in `pyproject.toml` (MediaPipe 1.2.0, NumPy 2.5.4, OpenCV-contrib 5.0.0.93, PyYAML 6.0.3, sounddevice 0.5.6). The image is based on `python:3.12-slim`; the container runs with the host user and the source is mounted read-only.

7 Limitations

- d depends on the hand-to-camera distance (Section 4).
- One hand is tracked; only the thumb–index gesture is mapped.
- The detection model is used as provided; no training or evaluation of the model itself is part of this project.
- End-to-end latency was not measured.

8 Conclusion

VolumeVision integrates a pretrained hand landmark model into a real-time pipeline, derives a control signal from a simple geometric feature, and separates behavior (`config.yaml`) from hardware (profiles), so the same code runs on different cameras and output devices.

References

[1] F. Zhang et al., *MediaPipe Hands: On-device Real-time Hand Tracking*, arXiv:2006.10214, 2020.

[2] C. Lugaresi et al., *MediaPipe: A Framework for Building Perception Pipelines*, arXiv:1906.08172, 2019.